affirmoTM tutorial 2: input files

Each input file: its layout, its rules, and how to look at it

2026-10-14

Table of contents

1 The input files 1

2 Transition matrices 2

3 Cohort 3

4 Entry states 4

5 Costs 4

6 Utilities 5

7 Intervention cost 5

8 Values that change over time: a `year` column 6

9 Costs and utilities that differ between patients 7

This tutorial starts from a fresh analysis folder with the example input files (see tutorial 1):

```
library(affirmoTM)
library(data.table)  # for fread(), fwrite() and the other table functions used below
```

```
tm_example_inputs()
```

1 The input files

All inputs live in `inputs/`, with fixed names. Your own files must use the same names and layouts.

File	What it holds	Read with
`settings.yaml`	Run settings (tutorial 3)	`tm_read_settings()`
`transition_matrix_usual_care.csv`, `transition_matrix_iabc.csv`	Yearly transition probabilities, one matrix per arm	`tm_read_matrices()`
`cohort.csv`	The patients	`tm_read_cohort()`
`entry_states.csv`	The chance of each state in a patient's first year	`tm_read_entry_states()`
`costs.csv`	The cost of a year in each state, per arm	`tm_read_costs()`

File	What it holds	Read with
`utilities.csv`	The QALY weight of a year in each state, per arm	`tm_read_utilities()`
`intervention_cost.csv`	The cost of delivering iABC, by year after entry	`tm_read_intervention_cost()`

Each reader checks its file and stops with a message naming the file and the problem. `tm_run()` reads and checks every file before it runs anything.

```
settings <- tm_read_settings()
```

2 Transition matrices

```
matrices <- tm_read_matrices(settings)
matrices$usual_care
```

```
<tm_matrix> transition matrix (source: user)
  Strata:         agegrp (9), sex (2), mm_cluster (7) -> 126 strata
  Living states:  13 - NoEvent, AB, AHS, AIS, PB, PHS, PIS, AHS_AB, AIS_AB, PB_AHS, PB_AIS, PS_AB, PS_PB
  Death states:   6 - CurrentDeath, AB_Death, AHS_Death, AIS_Death, AHS_AB_Death, AIS_AB_Death
  Years:          the same rows every year
  Draws:          none (point values)
```

The probabilities are in `probs`: one row per stratum (age group, sex, cluster) and current state (`from`), and one column per state, holding the probability of being in that state next year.

```
usual_care <- matrices$usual_care$probs
states <- matrices$usual_care$states
usual_care[1:3, 1:9]
```

```
   agegrp    sex  mm_cluster    from   NoEvent          AB         AHS         AIS
   <char> <char>      <char>  <char>     <num>       <num>       <num>       <num>
1:  70-74    men Unspecified NoEvent 0.9693759 0.004641617 0.002450905 0.006487908
2:  75-79    men Unspecified NoEvent 0.9571437 0.005880312 0.002995982 0.008038839
3:  65-69    men Unspecified NoEvent 0.9753106 0.003494647 0.001866399 0.005250628
      PB
   <num>
1:     0
2:     0
3:     0
```

The rules, and a quick way to look at each one:

- **Every row sums to 1.**

  ```
  usual_care[, range(rowSums(.SD)), .SDcols = states]
  ```

  ```
  [1] 1 1
  ```

- **Every stratum has a row for every state** (19 here):

  ```
  usual_care[, .N, by = .(agegrp, sex, mm_cluster)][, range(N)]
  ```

  ```
  [1] 19 19
  ```

- **Every state has its own rows, death states included.** A death state's rows go back to the same state with probability 1; that is how the package recognises death states. For example:

```
usual_care[from == "CurrentDeath", unique(CurrentDeath)]
```

```
[1] 1
```

- **The stratifier columns are the ones listed under `strata` in `settings.yaml`.** The age-group column must be called `agegrp`: it is the only stratifier that changes as patients age. Age groups are written as ranges, without gaps:

```
unique(usual_care$agegrp)
```

```
[1] "70-74"   "75-79"   "65-69"   "80-84"   "85-89"   "90-94"   "95-99"   "100-104"
[9] "105-109"
```

- **Both matrices have the same states and the same strata.**
- **The probabilities themselves are your responsibility.** The package checks the layout, not the numbers. When `tm_run()` starts, it lists the death states it found; check that no living state is among them.

3 Cohort

```
cohort <- tm_read_cohort(settings, matrices)
cohort
```

```
        id entry_cycle   age agegrp    sex       mm_cluster
     <int>       <int> <int> <char> <char>           <char>
  1:     1           0    65  65-69    men Neuropsychiatric
  2:     2           0    65  65-69    men Neuropsychiatric
  3:     3           0    65  65-69  women          Complex
  4:     4           0    65  65-69  women              Eye
  5:     5           0    65  65-69  women   Cardiovascular
 ---
4996:  4996           9    94  90-94  women      Unspecified
4997:  4997           9    95  95-99  women          Complex
4998:  4998           9    96  95-99  women              Eye
4999:  4999           9    97  95-99  women          Complex
5000:  5000           9    99  95-99  women              MSK
```

Column	Meaning
`id`	A unique patient id
`age`	Age in whole years in the patient's entry cycle (not at cycle 0)
`sex`, `mm_cluster`	One column for each stratifier other than `agegrp`
`entry_cycle`	The cycle in which the patient enters; 0 is the first year
`agegrp`	Optional: if it is missing, it is worked out from `age`

Patients can enter in different cycles:

```
cohort[, .N, keyby = entry_cycle]
```

```
Key: <entry_cycle>
    entry_cycle     N
          <int> <int>
 1:           0  2664
 2:           1   245
```

```
 3:           2   272
 4:           3   236
 5:           4   240
 6:           5   272
 7:           6   249
 8:           7   251
 9:           8   260
10:           9   311
```

The rules:

- ages lie within the matrices' age groups, and every value of `sex` and `mm_cluster` appears in the matrices;
- the matrices have rows for every stratum a patient reaches as they age;
- patients who enter in cycle `horizon` or later are left out, with a warning.

4 Entry states

The chance of each state in a patient's entry year, one row per stratum. Both arms start from the same entry states.

```
entry_states <- tm_read_entry_states(settings, matrices)
entry_states[agegrp == "75-79" & sex == "women" & mm_cluster == "Cardiovascular"]
```

```
   agegrp    sex     mm_cluster   NoEvent    AB         AHS        AIS          PB
   <char> <char>         <char>     <num> <num>       <num>      <num>       <num>
1:  75-79  women Cardiovascular 0.9011299     0 0.004708098 0.01506591 0.005649718
           PHS        PIS AHS_AB AIS_AB PB_AHS PB_AIS PS_AB PS_PB CurrentDeath
         <num>      <num>  <num>  <num>  <num>  <num> <num> <num>        <num>
1: 0.003766478 0.01506591      0      0      0      0     0     0   0.05461394
```

Every row sums to 1, and every patient's entry stratum needs a row:

```
entry_states[, range(rowSums(.SD)), .SDcols = setdiff(names(entry_states), settings$strata)]
```

```
[1] 1 1
```

5 Costs

The cost of a year spent in each state, one column per arm. Every state needs a row, death states included (the cost of the year of death). Costs are 0 or more.

```
costs <- tm_read_costs(settings, matrices)
costs
```

```
          state usual_care  iabc
         <char>      <num> <num>
 1:      NoEvent       2000  2000
 2:           AB       8000  8000
 3:          AHS      20000 20000
 4:          AIS      15000 15000
 5:           PB       3000  3000
 6:          PHS       5000  5000
 7:          PIS       4000  4000
 8:       AHS_AB      27000 27000
 9:       AIS_AB      22000 22000
10:       PB_AHS      22000 22000
11:       PB_AIS      17000 17000
```

```
12:        PS_AB      12000 12000
13:        PS_PB       6000  6000
14: CurrentDeath       3000  3000
15:     AB_Death       7000  7000
16:    AHS_Death      15000 15000
17:    AIS_Death      12000 12000
18: AHS_AB_Death      20000 20000
19: AIS_AB_Death      18000 18000
```

- **Entry year:** both arms are charged the `usual_care` cost, because both arms are in the same state. The `iabc` cost applies from the year after entry.
- **The `iabc` column:** use it for differences in care that depend on the state, such as medication. The cost of delivering the programme itself goes in `intervention_cost.csv`.

6 Utilities

The QALY weight of a year spent in each living state, one column per arm. Utilities are at most 1. Death states count 0 in the year of death, so they can be left out. In the entry year both arms take the `usual_care` utility.

```
utilities <- tm_read_utilities(settings, matrices)
utilities
```

```
          state usual_care  iabc
         <char>      <num> <num>
 1:      NoEvent       0.75  0.75
 2:           AB       0.65  0.65
 3:          AHS       0.50  0.50
 4:          AIS       0.55  0.55
 5:           PB       0.70  0.70
 6:          PHS       0.60  0.60
 7:          PIS       0.65  0.65
 8:       AHS_AB       0.40  0.40
 9:       AIS_AB       0.45  0.45
10:       PB_AHS       0.47  0.47
11:       PB_AIS       0.52  0.52
12:        PS_AB       0.55  0.55
13:        PS_PB       0.60  0.60
14: CurrentDeath       0.00  0.00
15:     AB_Death       0.00  0.00
16:    AHS_Death       0.00  0.00
17:    AIS_Death       0.00  0.00
18: AHS_AB_Death       0.00  0.00
19: AIS_AB_Death       0.00  0.00
```

7 Intervention cost

The cost of delivering iABC to one patient in each year after their entry (`year` 1 is the first year after entry), one row for every year from 1 to `horizon - 1`. It is the same whatever the patient's state, and is charged to iABC patients who are alive in that year.

```
intervention <- tm_read_intervention_cost(settings)
intervention
```

```
    year  cost
   <int> <num>
1:     1    50
2:     2     0
```

```
3:     3     0
4:     4     0
5:     5     0
6:     6     0
7:     7     0
8:     8     0
9:     9     0
```

8 Values that change over time: a `year` column

A matrix, `costs.csv` or `utilities.csv` can change with the years since a patient's entry. Add a `year` column (1 is the first year after entry) and give a full set of rows for every year from 1 to `horizon - 1`. In a matrix, the rows for year k drive the move into year k after entry.

For example, an iABC effect that lasts one year: iABC's own probabilities in year 1, and usual care's in years 2 to 9.

```
usual_care_file <- fread("inputs/transition_matrix_usual_care.csv")
iabc_file <- fread("inputs/transition_matrix_iabc.csv")
later_years <- usual_care_file[rep(seq_len(.N), 8)][, year := rep(2:9, each =
 ↪  nrow(usual_care_file))]
iabc_by_year <- rbind(iabc_file[, year := 1L], later_years)
iabc_by_year[, .N, keyby = year]
```

```
Key: <year>
    year     N
   <int> <int>
1:     1  2394
2:     2  2394
3:     3  2394
4:     4  2394
5:     5  2394
6:     6  2394
7:     7  2394
8:     8  2394
9:     9  2394
```

```
fwrite(iabc_by_year, "inputs/transition_matrix_iabc.csv")
```

The package reads it as a matrix that changes by year:

```
tm_read_matrices(settings)$iabc
```

```
<tm_matrix> transition matrix (source: user)
  Strata:         agegrp (9), sex (2), mm_cluster (7) -> 126 strata
  Living states:  13 - NoEvent, AB, AHS, AIS, PB, PHS, PIS, AHS_AB, AIS_AB, PB_AHS, PB_AIS, PS_AB, PS_PB
  Death states:   6 - CurrentDeath, AB_Death, AHS_Death, AIS_Death, AHS_AB_Death, AIS_AB_Death
  Years:          by year since entry, 1 to 9 (year column)
  Draws:          none (point values)
```

The result, next to the example's, with 20 runs each:

```
twenty_runs <- settings
twenty_runs$n_runs <- 20
one_year <- tm_run(twenty_runs)$comparison
tm_example_inputs(overwrite = TRUE)  # put the example files back
every_year <- tm_run(twenty_runs)$comparison
```

```
rbind(cbind(effect = "every year", every_year[, .(delta_cost, delta_qaly, icer, quadrant)]),
      cbind(effect = "first year only", one_year[, .(delta_cost, delta_qaly, icer, quadrant)]))
```

```
           effect delta_cost  delta_qaly      icer             quadrant
           <char>      <num>       <num>     <num>               <char>
1:      every year -104.67869 0.004967325 -21073.45             dominant
2: first year only   10.75372 0.001178643   9123.81 costlier, more QALYs
```

9 Costs and utilities that differ between patients

`costs.csv` and `utilities.csv` can also have stratifier columns, any of the `strata` in `settings.yaml`. Each patient-year then uses the patient's stratum in that year: their current age group, as they age, and their sex and cluster. Every stratum needs a row for every state.

For example, utilities that fall with age and are lower for women, and costs that differ by cluster. The factors are invented for this example.

```
utilities_file <- fread("inputs/utilities.csv")
age_groups <- unique(usual_care$agegrp)
by_age_sex <- utilities_file[, .(agegrp = age_groups), by = .(state, usual_care, iabc)]
by_age_sex <- by_age_sex[, .(sex = c("men", "women")), by = .(state, usual_care, iabc, agegrp)]
by_age_sex[, factor := 1 - 0.03 * (as.integer(sub("-.*", "", agegrp)) - 65) / 5 - 0.02 * (sex
 ↪  == "women")]
by_age_sex[, `:=`(usual_care = usual_care * factor, iabc = iabc * factor)][, factor := NULL]
by_age_sex[state == "NoEvent" & agegrp %in% c("65-69", "85-89")]
```

```
     state usual_care  iabc agegrp    sex
    <char>      <num> <num> <char> <char>
1: NoEvent      0.750 0.750  65-69    men
2: NoEvent      0.735 0.735  65-69  women
3: NoEvent      0.660 0.660  85-89    men
4: NoEvent      0.645 0.645  85-89  women
```

```
costs_file <- fread("inputs/costs.csv")
cluster_factor <- c(Cardiovascular = 1.2, Complex = 1.5, Eye = 1.0, Metabolic = 1.1, MSK = 1.0,
                    Neuropsychiatric = 1.2, Unspecified = 1.0)
by_cluster <- costs_file[, .(mm_cluster = names(cluster_factor)), by = .(state, usual_care,
 ↪  iabc)]
by_cluster[, `:=`(usual_care = usual_care * cluster_factor[mm_cluster], iabc = iabc *
 ↪  cluster_factor[mm_cluster])]
by_cluster[state == "NoEvent"]
```

```
     state usual_care  iabc       mm_cluster
    <char>      <num> <num>           <char>
1: NoEvent       2400  2400   Cardiovascular
2: NoEvent       3000  3000          Complex
3: NoEvent       2000  2000              Eye
4: NoEvent       2200  2200        Metabolic
5: NoEvent       2000  2000              MSK
6: NoEvent       2400  2400 Neuropsychiatric
7: NoEvent       2000  2000      Unspecified
```

```
fwrite(by_age_sex, "inputs/utilities.csv")
fwrite(by_cluster, "inputs/costs.csv")
```

The result, next to the example's:

```
by_stratum <- tm_run(twenty_runs)$comparison
tm_example_inputs(overwrite = TRUE)  # put the example files back
rbind(cbind(values = "the same for everyone", every_year[, .(delta_cost, delta_qaly, icer,
 ↪  quadrant)]),
      cbind(values = "by age, sex and cluster", by_stratum[, .(delta_cost, delta_qaly, icer,
       ↪  quadrant)]))
```

```
                  values delta_cost  delta_qaly      icer quadrant
                  <char>      <num>       <num>     <num>   <char>
1:   the same for everyone  -104.6787 0.004967325 -21073.45 dominant
2: by age, sex and cluster  -124.8150 0.004448130 -28060.11 dominant
```